lack thereof, which has an impact on the execution of the DT. To tackle this issue, the DT was extended with time discrepancy detection capabilities, allowing it to degrade to DS when there was a degradation in communication as discussed in Sect. 1.2.2 [8]. Finally, the DT was extended further with a model swapping capability, enabling it to swap models at run-time under configurable conditions [2].

12.4.3 Summary and Future Work

The DR case-study has shown proof-of-concept DTs for a mobile robot, covering functionalities such as: (i) parallel operation, where the DT continuously predicts where the robot should be as well as having access to the actual position data, thus laying the groundwork for monitoring and fault detection; (ii) fault injection, where different components in the DT can be injected with faulty values allowing different kinds of experiments with hardware in the loop, as close to the operational stage as possible; (iii) model swapping, where the DT is able during run-time to swap components, given that certain conditions – specified by the user – are satisfied.

The main challenge is related to the simulation-reality gap, and the development of high-fidelity models. Future work will focus on this aspect, as well as enhancing the DT to deal with a multi-robot system, providing functionalities like the evaluation and mitigation of collision zones between robots during deployment. Additionally, fault detection and diagnosis shall be explored, with relevant techniques to be implemented in the DT such that scenarios where, e.g., a robot gets a flat tire, can be identified and handled properly. Finally, the fault injection framework could be extended to allow for fault injection at the DT and the PT levels simultaneously, allowing for richer experimental setups for testing the adopted fault tolerance mechanisms. Fault injection could also be used in the context of what-if simulations, by providing possibly useful insights to the fault diagnostics.

12.5 The Flex-cell

The flex-cell is a manufacturing cell composed of two robotic arms used for cooperative assembly. A key feature is that the two arms are different and from diverse manufacturers. As an academic DT engineering case study, it provides opportunities for researchers to explore:

- composable DTs;
- cooperative systems with synchronous motions and coupled models;
- multiple abstraction levels of attributes and operations of both robots and grippers;

- complex system with kinematic and dynamic models for the behavioural aspects in the cooperative setup; and
- multiple robotics applications that include different operations, such as pick-and-place, peg-in-hole, optimisation in assembly, payload estimation, online recalibration and collision detection/avoidance.

As shown in Fig. 12.23, the PT is composed of four main assets: a Kuka LBR iiwa 7 robotic arm; a UR5e robotic arm; an OnRobot RG6 gripper; an OnRobot 2FG7 gripper. The flex-cell DT's purpose is to assist the PT with the features listed above. The applications that are run on the flex-cell are based on a 2D cooperative assembly where the scope is related to robot positioning. For this. the real workspace (x, y, z) of the flex-cell plate is discretised to (X, Y, Z), where the discrete positions use the plate holes, creating a grid of 16x24 holes separated by 5mm each; the Z axis is also discretised using a 5mm distance between points.

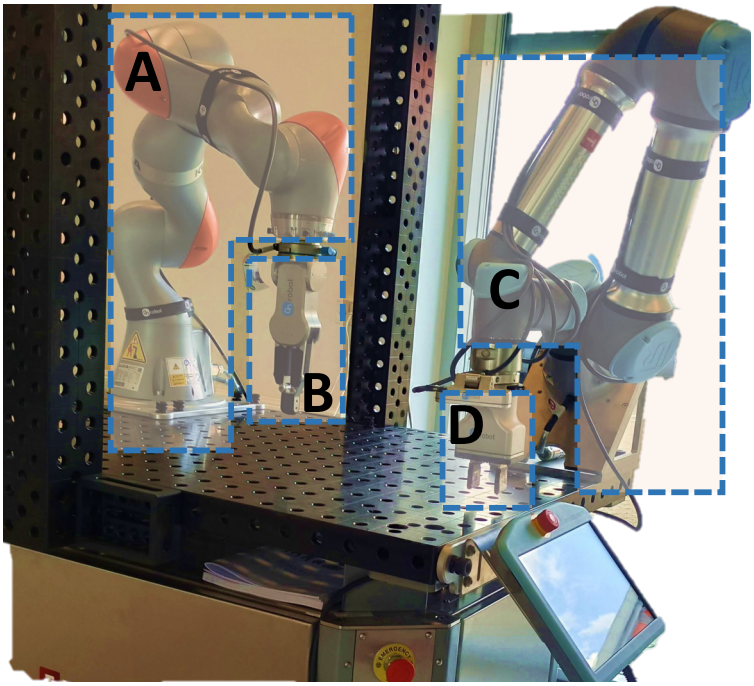

Fig. 12.23: Flex-cell and its components: robotic arms Kuka LBR iiwa 7 (A) and UR5e (C), and grippers OnRobot RG6 (B) and 2FG7 (D).

12.5.1 Flex-cell DT Overview

The flex-cell DT has been approached from a composable perspective using the modelling approach proposed in [11], therefore, the four main assets, i.e., robotic arms and grippers, have a sub-DT representation in the final composed DT. More precisely, using such a modelling approach, each asset is represented as a DT defined by *Attributes*, *Operations*, *Behaviours*, and *Relationships*. In this approach, composition is achieved by the particular relationship *isComposedOf*, which requires the subject (the parent, i.e., the composed DT) and the objects (children, i.e., smaller DTs). For the flex-cell case study, the flex-cell DT instance *isComposedOf* the DT instances of the Kuka LBR iiwa 7, the UR5e, the gripper OnRobot RG6, and the gripper OnRobot 2FG7.

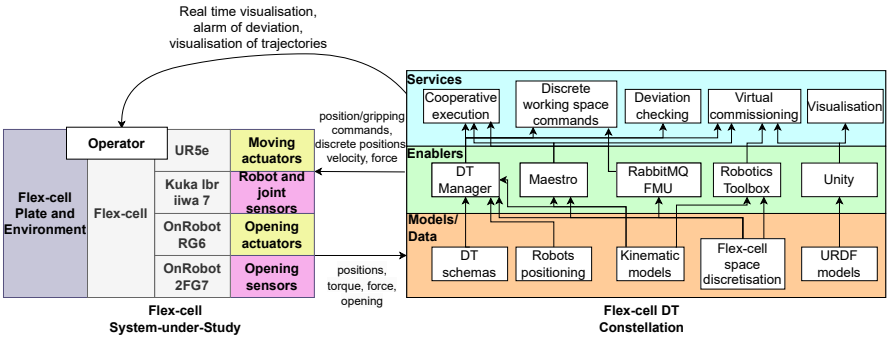

Fig. 12.24: Constellation of the flex-cell DT.

Fig. 12.24 illustrates the constellation of the flex-cell DT. Only the two robotic arms are provided with simulation models, while the grippers only contain the static and structural information of their PTs. Moreover, the simulation models for the robotic arms only contain the kinematic models; these are created using the Robotics Toolbox [1] and embedded into FMUs via the UniFMU tool [23].

For the co-simulation of the flex-cell we use Maestro and the RabbitMQFMU [7]. The purpose of the co-simulation and the RabbitMQFMU is to enable synchronised motions of both robots in the real and virtual settings. In this sense, RabbitMQFMU replicates the commands of the actual controller to the robots during runtime. Fig. 12.25 shows a representation of the co-simulation setup of the flex-cell to execute synchronous and cooperative commands in the flex-cell system.

We use the *DT Manager* approach proposed in [24] to handle the connectivity between PT and DT and the game engine Unity[6] along with Unified Robotics Description Format (URDF) models for the robots and grippers to generate the

[6] https://unity.com/

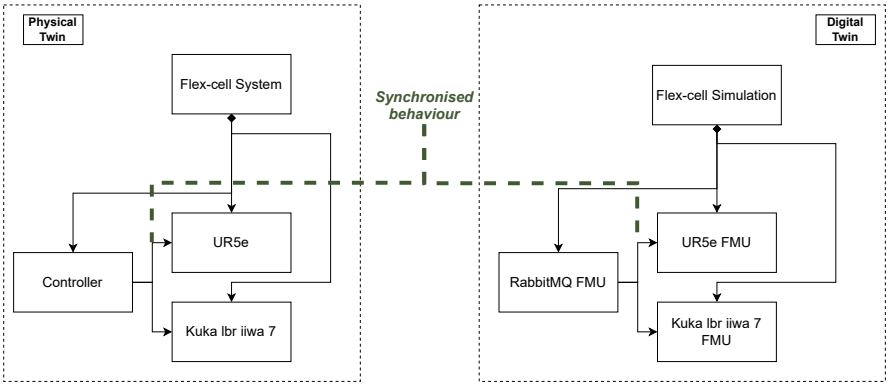

Fig. 12.25: Co-simulation setup of the flex-cell DT.

visual representation of the DT of the flex-cell shown in Fig. 12.26. The DT Manager provides an interface to administer DTs through (i) abstracting PTs and DTs at the same abstraction level, i.e., both are handled as *Twins* and (i) providing the methods to read, write and execute operations on PTs and simulation units straightforwardly.

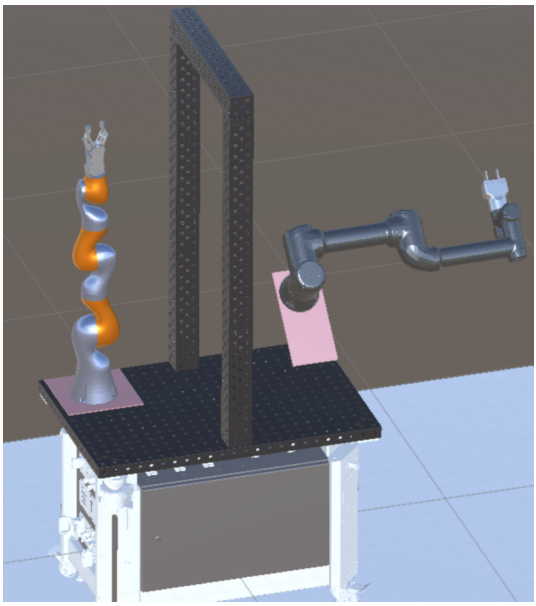

Fig. 12.26: Flex-cell graphical representation in Unity.

12.5.2 Fundamental Characteristics

This section presents the flex-cell case study through the 14 fundamental DT characteristics as introduced in Sect. 12.2.

12.5.2.1 C1: System-under-study

The *flex-cell system* itself is the manufacturing cell PT described above with the two robotic arms with attached grippers. The *environment* is a plate with 16×24 holes where there are bricks and pins/pegs to place on the holes. The environment also has a safety system, which is activated in case of a collision or an abnormal condition. The safety system also has a proximity sensor which is enabled to stop the robots if anyone gets close to the working space. Each robotic arm has a "teach pendant" allowing an *agent*, a human operator, can manipulate the robots or set them to work with remote control for automatic execution. The sequences to assemble a shape are executed remotely from a PC by the human operator.

12.5.2.2 C2: Acting Components

The acting components in the system are the two robotic arms, the two grippers, and the safety system. These are provided with network interfaces, which allow the execution of actions with their particular APIs, either through Transmission Control Protocol (TCP) for the robotic arms or ModbusTCP for the grippers. The commands sent to the robotic arms are processed by their internal controllers to perform different motion types, such as cartesian, joint, linear, and circular movements, and update their parameters, such as speed, acceleration, etc. The teach pendants can also be used for manually moving the robots and updating their configurations. Similarly, the commands to the grippers are processed by their internal controllers, where the human operator can update the default values of width/opening of the gripper and the gripping force. The safety system simply stops the robots if there is a collision or a person tries to get close to the working space.

12.5.2.3 C3: Sensing Components

Regarding the sensors, the robotic arms contain several internal sensors from which it is possible to collect data, such as current position, target position, torque, current, voltage, speed, acceleration and so on. Similar to the acting components, the assets are provided with network interfaces to allow the collection of data with their particular APIs, either through TCP (arms) or ModbusTCP (grippers). Some of the variables for the robot contain their angular and cartesian value, and there are some variables

that are specific for each joint and some that are for the whole robotic arm. We can collect 117 and 31 variables for the UR5e robot and the Kuka lbr iiwa 7 robot respectively, although not all of them are used. For the grippers, it is possible to collect force and width/opening values. The safety system has a proximity sensor and internal flags in case any of the robotic arms collide.

12.5.2.4 C4: Multiplicities

The flex-cell DT is addressed with the composable modelling approach presented in [11]. Therefore, the DT-enabled system contains seven DT instances, namely, one DT per asset in the flex-cell system, two intermediate composition of robotic arm + gripper (one for the Kuka LBR iiwa 7 + OnRobot RG6 and one for the UR5e + OnRobot 2FG7), and a final composition of the flex-cell system which composes the two intermediate compositions. Through this modelling approach, the multiple DTs and their corresponding PTs can be handled at the higher hierarchy level, in this case, the flex-cell DT, by disaggregation. Although there are multiple instances of DTs, there are no multiplicities of the same DT class since the flex-cell system is built with different assets.

12.5.2.5 C5: Data Transmitted

The transmission of data between PT and DT and vice-versa is carried out by the DT Manager approach [24], which provides the methods to read, write, and execute operations on the endpoints of PTs and DTs. More precisely, reading from an endpoint is possible through the method `getAttributeValue(s)`, writing to an endpoint is possible through the method `setAttributeValue(s)`, executing operations on an endpoint is possible through the method `executeOperation`. These three methods enable the bidirectional communication between PTs and DTs as follows: The method `getAttributeValue(s)` provides the interface to read data from the twins and the methods `setAttributeValue(s)` and `executeOperation` provide the interface to write data to and execute operations on the twins. In terms of sending data to PTs, the method `setAttributeValue(s)` is preferred to send *insights*, such as parameter updates, whereas the method `executeOperation` is preferred to execute direct actions or commands. These methods work on both PTs and DTs since both are categorised at the same abstraction level, so they are covered by the same class. It is therefore possible to use these methods on both equally, depending on the variables and operations they actually have. For the PT, any of the available variables can be transmitted, and the same with the different available move commands. For the DT, this is limited to what the models support and which variables they can provide. With the kinematic models that we use, it is possible to

get position variables and execute angular and cartesian motions. However, speed, acceleration and torque data are not available for those specific models.

12.5.2.6 C6: Insights/Actions

As *actions*, the DT Manager can perform moving commands on both the PT and the DT. The DT provides the joint angles to the PT when the move commands are based on the flex-cell discrete working space. Similarly, if DT services are added on top of the DT Manager, *insights* are generated, such as for example, deviation checking or discrete collision detection in the simulation. The visualisation with Unity (see Fig. 12.26) also provides insight into how the trajectory will be performed by the real robots. Refer to Sect. 8.4 for further information on visualisation.

12.5.2.7 C7: Services

The services that the DT currently provides for the flex-cell are:

- *Cooperative execution* that can perform moving commands on both robots on both physical and virtual versions synchronously.
- *Trajectory visualisation* to help the user understand how the robots are going to move for a certain move command in the flex-cell working space (see Sect. 8.4 for further information on visualisation).
- *Discrete flex-cell working space-related commands* that translate the target (X, Y, Z) position in the flex-cell to joint angles of the robots.
- *Deviation checking* is used to compare that the physical and the virtual robots are moving similarly as theoretically expected (see Sect. 9.4 for further information on *anomaly detection*).
- *Virtual commissioning* where the DT is decoupled from the PT to perform *what-if* experiments and validate on the virtual flex-cell before deployment to the physical one (see Sect. 10.2 for further information on *what-if simulations*).

12.5.2.8 C8: Enablers

The following *enablers* support the flex-cell DT services:

- The DT Manager to provide the access and interfaces to PT and DT, and to the DT services.
- The Robotics Toolbox and UniFMU to encapsulate the kinematic models of the robotic arms.
- The urinterface, kukalbrinterface, ModbusTCP, and MQTT to access the physical robots and grippers and bridge the communication to external networks.

- Maestro and the RabbitMQFMU to perform the synchronous and coupled co-simulations.
- Unity, ZeroMQ and URDF are joint enablers for displaying visualisations.

12.5.2.9 C9: Models and Data

The flex-cell DT contains the following:

Data models. There is a data model for each robotic arm and gripper, which handles the *attributes* and *operations* of the DT using schemas. The data models are also extended with the definitions for *behaviours* and *relationships* as presented in [11].

Structural models. There is a structural model for each asset in the flex-cell system, which are used for visualisation. These models use the URDF format, which contains the descriptions of the rigid body, links, and joints.

Behavioural models. Additionally, for the robotic arms there are behavioural models as follows: (i) kinematic models using the Denavit-Hartenberg parameters; (ii) dynamic models with the Newton-Euler formulation (not being used). There is also a calibrated dynamics model for the UR5e robotic arm generated with the AURT tool [27]. The data of interest is contained in the DT schemas and the available outputs from the behavioural models in use, which are related to the positioning of the robotic arms.

12.5.2.10 C10: Constellation

We can think of the services offered in terms of vertical slices through the layered structure shown in Fig. 12.24. In a scenario for *cooperative execution*, *deviation checking*, and implicitly *discrete working space commands*, Fig. 12.27 shows one of the experiments of running coupled cooperative motions along with the co-simulation. The figure shows the joint positions over time of the two robots. The experiment consists of moving the robots between two different discrete waypoints. The simulation is initialised from zero values, but the real system starts from its previous position. Therefore the models and the real robots converge after the models have reached the first target position.

In the scenario for *virtual commissioning and visualisation*, Fig. 12.26 shows the graphical representation of the flex-cell system in Unity. The Unity app embeds the URDF models and the incoming messages from either the flex-cell simulation or real flex-cell are translated into motions, enabling the visualisation of the system as a DT. When using the simulated flex-cell, this setup is also used for virtual commissioning in which it is possible to see the trajectories of the robotic arms before deployment.

In the scenario for *deviation checking*, a monitoring function is used that checks for deviations periodically. This function takes as inputs the DT and PT, the variable

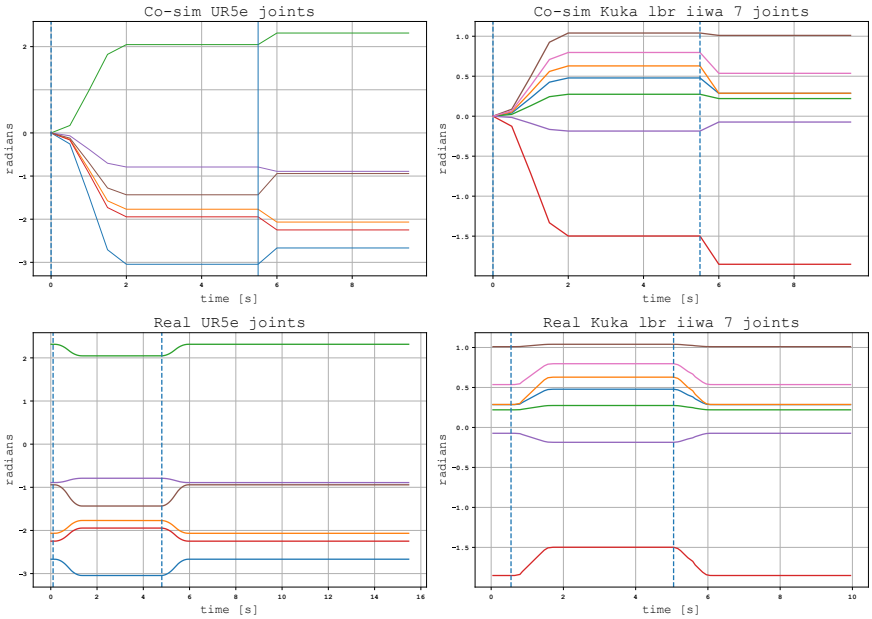

Fig. 12.27: Plot of the joint positions of the robotic arms in the cooperative co-simulation setting. The vertical dashed lines indicate the invocation of moving commands. Each coloured line represents the angle of one of the robot's joints.

of interest, and the tolerance, and returns *true* when there is a deviation in the variable between the two objects given the tolerance, otherwise *false*.

12.5.2.11 C11: Time-Scale

The DT for the flex-cell operates at different rates:

- **Slower-than-real-time:** The services that need to go over the DT Manager and generate an output based on PT and DT data.
- **Real-time:** Control commands, data recording, and visualisation.
- **Faster-than-real-time:** For virtual commissioning, and so the DT is decoupled from the PT to run independent simulations and scenarios.

12.5.2.12 C12: Fidelity Considerations

Medium- to high-fidelity models are required to deliver good performance for the provided services, especially those related to the positioning of the robotic arms.

Note that insufficient fidelity may cause the robots to collide with and damage static objects like the flex-cell structure, with other dynamic objects like the other robotic arm or grippers, and with humans.

As downsides, there are the following:

- The kinematic models do not include the grippers as attached on the end-effector of the robotic arms.
- The trajectory generation with the kinematic models provides certain time behaviour based on an assumed motion speed. However, it does not consider the actual dynamics of the robots, and so, it can have medium to high time offsets. Higher fidelity models for the actual dynamics may overcome this problem.

12.5.2.13 C13: Life-cycle Stages

The services provided on top of the flex-cell DT include the *design*, *manufacturing*, and *service* life-cycle phases for DT applications proposed in Liu et al. [25]. The manufacturing and service phases are administered by the DT Manager and the DTaaS platform (see Sect. 11.7). For the design phase, the virtual commissioning service of the DT is used, i.e., decoupled from the PT, to validate changes to the applications to be run in the physical system.

12.5.2.14 C14: Evolution

The flex-cell case study has evolved considerably during development. Some of the evolution steps are reported in [11, 13]. The most representative milestones are as follows:

1. The DT schema-based object models for robotic arms and grippers were designed.
2. The object models were coupled to the DT Manager architecture.
3. The visualisation with Unity and the URDF models were created.
4. The composable representation of the flex-cell was established.
5. The kinematic and dynamic models for the robotic arms were designed.
6. The space and mapping model for the flex-cell was created.
7. The execution of online PT-to-DT movements with individual non-cooperative motions was achieved.
8. The first experiments of applications for the flex-cell discrete workspace using the DT as the virtual commissioning mechanism were successful.
9. The skill-based engineering approach was integrated to disaggregate the broader term *operations* into *device primitives*, *skills*, and *tasks*.
10. The kinematic models were embedded into FMUs.

11. The co-simulation with Maestro and RabbitMQFMU was added to provide simulation capabilities of the coupled cooperative motions.
12. Synchronous cooperative experiments are successful and used for deviation checking.
13. The flex-cell DT components are uploaded to the DTaaS platform (see Sect. 11.7 for further information on the DTaaS platform).

12.5.3 Summary and Future Work

The flex-cell case study provides a representative system for DT engineering in cooperative robotics, being composed of two robotic arms of different brands, with joints and grippers. It therefore gives an opportunity to explore the composition of DTs and models. The services for this case study are handled on its working space, i.e., the plate, mostly with a scope in robot positioning. There are multiple enablers for the modelling, communication and simulation aspects, such as the Robotics Toolbox and UniFMU for integrating the kinematic models of the robots, Maestro and RabbitMQFMU for running co-simulations, the DT Manager for administrating the interfaces and Unity and URDF for the visualisation components.

The most challenging aspect of this case study the integration of the models in a cooperative-synchronised setting. This is because there are multiple modelling approaches for robotic arms for different components, e.g., kinematics, dynamics, visualisation, programming. This makes it difficult to integrate all of them into a single DT if the scope is broad. Thus, it is better to narrow the scope to the applications that are needed, and hence choose the appropriate models for that scope. The current scope of the flex-cell is related to positioning, but it may be widened in the future to cover other relevant aspects.

The case study still evolves, mostly in terms of applications and services, such as collision detection, optimisation of assembly, and flexible and reconfigurable sequences. Additional models for other applications, such as payload estimation, are also possible evolution scenarios. The ongoing work of the flex-cell DT focuses on the generation of sequences for flexible manufacturing in a cooperative setting by integrating the skill-based programming approach [32].

12.6 The Research Vessel Gunnerus

The R/V Gunnerus, shown in Fig. 12.28, is a research vessel owned by the Norwegian University of Science and Technology (NTNU) that is used to support marine research and exploration. The vessel is equipped with the latest technology for a variety of research activities in biology, technology, geology, archaeology, oceanog-